

Laboratory Instructions

Interval Analysis and Moore Intervals

overview version for computer-based work

Duration: 90 minutes

Format: computer laboratory

Programming language: Python

The goal of this laboratory is to provide a practical introduction to the basics of interval arithmetic: from simple operations on intervals, through the dependency problem, to one iteration of the interval Newton method. The session has an **overview character**: the most important aspects are understanding the main ideas, interpreting the results, and carrying out short numerical experiments.

1. Learning objectives

After completing the exercise, the student should be able to:

- understand what an interval is as a description of a value together with uncertainty,
- perform basic operations on intervals,
- recognize that two algebraically equivalent formulas may produce different interval estimates,
- interpret the width of an interval as a measure of output uncertainty,
- understand the idea of interval contraction in the interval Newton method.

2. Work organization

Mode of work. The laboratory is completed individually or in pairs at computers. Every task should be executed, and the result should then be briefly **interpreted**. Code alone is not sufficient.

Required environment:

- Python 3.x,
- any editor or Jupyter Notebook,
- no external libraries.

Suggested time allocation:

Part	Scope	Time
I	Running the starter code and basic operations	15 min
II	Experiment: two representations of the same function	20 min
III	Influence of input interval width on the result	15 min
IV	One or several iterations of interval Newton	25 min
V	Measurement application and final conclusions	15 min

3. Starter code

Copy the following code and run it in your environment.

```
class Interval:
    def __init__(self, a, b):
        assert a <= b, "The left endpoint must be <= the right endpoint"
        self.a = a
        self.b = b

    def __repr__(self):
        return f"[{self.a}, {self.b}]"

    def width(self):
        return self.b - self.a

    def mid(self):
        return (self.a + self.b) / 2

    def __add__(self, other):
        return Interval(self.a + other.a, self.b + other.b)
```

```
def __sub__(self, other):
    return Interval(self.a - other.b, self.b - other.a)

def __mul__(self, other):
    vals = [
        self.a * other.a,
        self.a * other.b,
        self.b * other.a,
        self.b * other.b,
    ]
    return Interval(min(vals), max(vals))
```

Comment. In this laboratory we use a simplified implementation. We do not yet address outward rounding at the machine level, but we preserve the main idea: the result should contain all admissible values.

4. Laboratory tasks

Part I. Basics of interval arithmetic

Task 1. For the intervals

$$X = [-1, 2], \quad Y = [3, 5]$$

run the code:

```
X = Interval(-1, 2)
Y = Interval(3, 5)

print("X + Y =", X + Y)
print("X - Y =", X - Y)
print("X * Y =", X * Y)
print("width of X =", X.width())
print("midpoint of X =", X.mid())
```

To include in the report:

1. The results of all operations.
2. A one- or two-sentence interpretation of the multiplication result.
3. An explanation of what the midpoint and the width of an interval mean.

Part II. Dependency on the form of the expression

Consider the function

$$f(x) = x^2 - x.$$

Implement two algebraically equivalent forms:

```
def f1(X):
    return X * X - X

def f2(X):
    return X * (X - Interval(1, 1))
```

Task 2. For the interval $X = [1, 2]$, run:

```
X = Interval(1, 2)
print("f1(X) =", f1(X), "; width =", f1(X).width())
```

```
print("f2(X) =", f2(X), "; width =", f2(X).width())
```

Questions to answer:

1. Are the two results identical?
2. Which result is narrower?
3. Why can two equivalent formulas produce different interval estimates?

Interpretive hint. If the same variable appears several times, simple interval arithmetic treats these occurrences independently. This leads to an artificial widening of the result.

Part III. Influence of the width of the input interval

Task 3. Investigate how the result changes for increasingly narrow intervals:

```
intervals = [
    Interval(1, 2),
    Interval(1, 1.5),
    Interval(1, 1.1),
]

for X in intervals:
    print("X =", X)
    print(" f1(X) =", f1(X), "; width =", f1(X).width())
    print(" f2(X) =", f2(X), "; width =", f2(X).width())
    print()
```

Discuss:

1. What happens to the width of the result when the input interval is narrowed?
2. Does the difference between `f1` and `f2` decrease or increase?
3. Why is this important in practical numerical computations?

Part IV. Interval Newton - simplified version

Consider the equation

$$f(x) = x^2 - 2 = 0,$$

whose positive solution is $\sqrt{2}$.

First add the reciprocal and division functions:

```
def inv(X):
    assert not (X.a <= 0 <= X.b), "Cannot invert an interval containing zero"
    return Interval(1 / X.b, 1 / X.a)

def div(X, Y):
    return X * inv(Y)
```

Then implement the interval Newton operator:

```
def newton(X):
    m = X.mid()
    f_m = m * m - 2
    derivative = Interval(2 * X.a, 2 * X.b)
    return Interval(m, m) - div(Interval(f_m, f_m), derivative)
```

Task 4. For the initial interval $X^{(0)} = [1, 2]$, perform 3–5 iterations:

```
X = Interval(1, 2)

for i in range(5):
    N = newton(X)
    X = Interval(max(X.a, N.a), min(X.b, N.b))
    print(f"iteration {i+1}: X = {X}, width = {X.width()}")
```

To include in the report:

1. The sequence of intervals obtained in the iterations.
2. An observation of whether the interval contracts.
3. A brief explanation of why this method is useful in verified numerical computations.

Part V. Mini measurement application

Task 5. The side lengths of a rectangle were measured with uncertainty:

$$L = [0.999, 1.001], \quad W = [1.999, 2.001].$$

Compute the interval for the area:

```
L = Interval(0.999, 1.001)
W = Interval(1.999, 2.001)
A = L * W
print("Area A =", A)
print("Width of the result =", A.width())
```

Final questions:

1. What does the resulting interval for the area mean?
2. Why does the single value $A = 2$ not convey the full information?
3. In which engineering or scientific applications could this way of computing be useful?

5. What should be included in the submitted report

The report may take the form of a short PDF file or a Jupyter notebook exported to PDF. It should contain:

- the student's name,
- the results of all executed experiments,
- short interpretive answers for each task,
- a concise summary: *what was the main advantage of interval arithmetic, and what was its limitation.*

Minimum final summary. In 5–8 sentences, the student should address three issues: (1) what interval analysis provides, (2) where the overestimation comes from, and (3) where this method may be useful in practice.